Static Analysis of Spring PetClinic

Task 2 - SonarQube & jQAssistant Analysis

FPSS · TU Dresden · Group 15

Objectives

- Analyze code quality
- Analyze architecture
- Evaluate future-proof metrics
- Find strengths & weaknesses

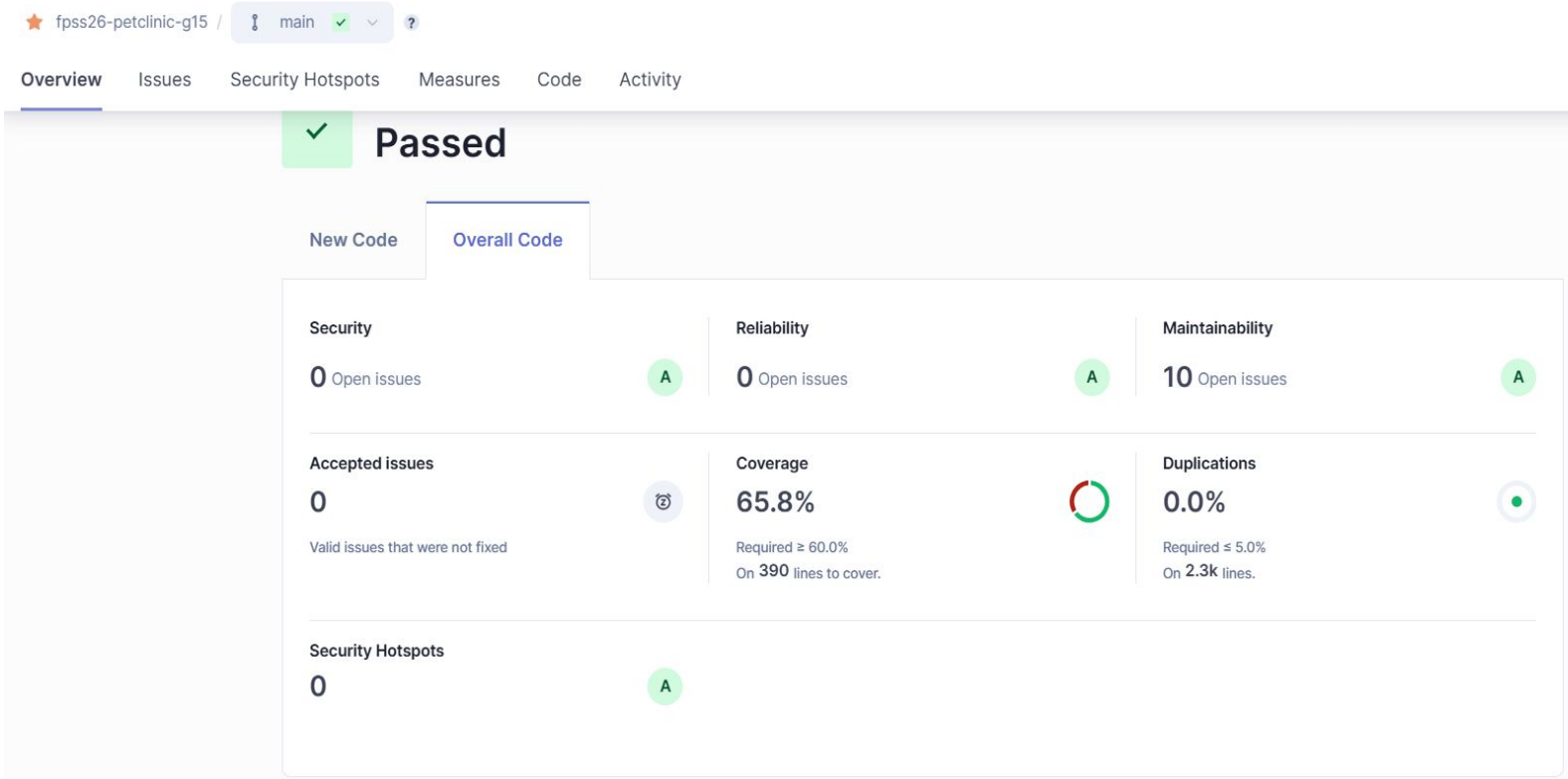
Tools Used

- SonarQube
- jQAssistant + Neo4j
- Cypher Queries
- GitHub CI/CD

Project

- Spring PetClinic
- Group 15

SonarQube Results Overview



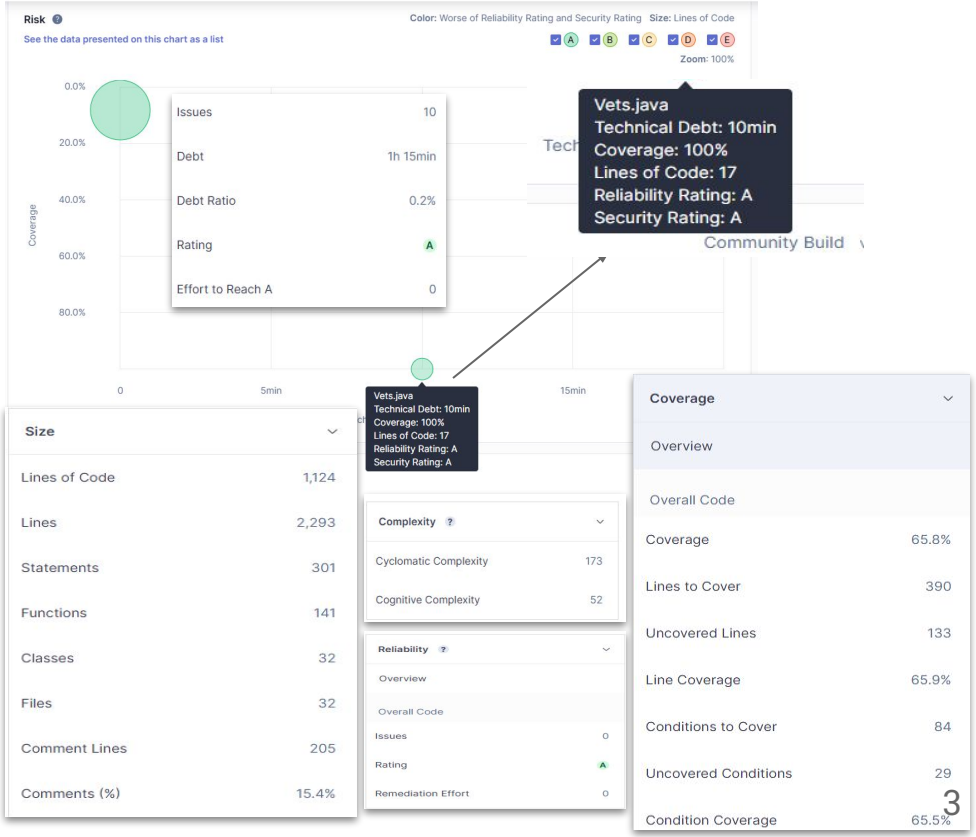
How Can SonarQube Detect Weaknesses?

WEAKNESSES DETECTED

- Code smells & maintainability issues
- Duplicated code blocks
- Low test coverage gaps
- Potential bugs
- Security vulnerabilities
- Technical debt accumulation

KEY BENEFITS

- Early detection of quality problems
- Continuous quality monitoring
- CI/CD pipeline integration
- Long-term maintainability support
- Cognitive complexity scoring
- Management-friendly debt metrics



Useful Quality Gates

A Quality Gate is an automated pass/fail checkpoint in the CI/CD pipeline that blocks a release if the code does not meet defined quality conditions.



Quality Gate	Threshold	Our Result	Why It Matters
Coverage	≥ 60%	65.8%	Close to threshold. Untested features reduce this over time.
Duplications	≤ 5%	0.0%	Zero duplication. Reduces maintenance effort and redundancy.
Security	A	A	Indicates no security issues were detected.
Reliability	A	A	Zero bugs detected. System behaves correctly.
Maintainability	Not worse than B	A	Only minor (10) code smells. Codebase remains easy to change and extend.
Hotspots Reviewed	0 hotspots or 100% reviewed	0 hotspots	No security hotspots required review.

Weaknesses of SonarQube

Limitations

- SonarQube cannot

- Verify business logic correctness
- Limited ability to detect performance bottlenecks
- Cannot reliably detect runtime memory leaks
- Limited architectural analysis compared to specialized tools (e.g., jQAssistant)
- Static rules may not fully understand framework-specific design intentions

- False Positives:

- Some issues may not be actual problems.
- Example: TODO comments are reported as maintainability issues.

- Coverage Limitation: High coverage does not guarantee bug-free software.

- Metric Gaming:

- Teams can write trivial tests just to hit coverage thresholds
- Quality gate passing does not mean code is actually well-tested

- Small Project Limitation:

- Our PetClinic project is too small to trigger many meaningful SonarQube insights
- Most metrics show 0 issues — not because the code is perfect, but because the codebase is small
- SonarQube becomes truly useful only at larger scale with many contributors

- Context-Insensitive Rules:

- Rules are generic — no distinction between critical and non-critical code
- A violation in a utility class is treated the same as one in core business logic

Future-Proof Software Metrics

Metrics derived from FPSS lecture concepts and measured via SonarQube and jQAssistant.

Metric	Tool	Future-Proof Aspect
Reuse Factor	jQAssistant	Reusability
Coupling Analysis	jQAssistant	Changeability
Technical Debt Ratio	SonarQube	Maintainability
Duplication Ratio	SonarQube	Architecture Quality
Coverage	SonarQube	Reliability of Evolution

Metrics Selected

The selected metrics are based on:

- Reuse Factor
- Coupling Analysis
- Technical Debt Ratio
- Duplication Ratio
- Coverage

Future-Proof Qualities Evaluated

- Reusability
- Maintainability
- Changeability
- Technical Debt
- Architecture Quality
- Encapsulation

Reuse Factor Analysis · Q-1

```
MATCH (c:Type:Class)
WITH collect(c) AS allClasses

MATCH (a:Type:Class)-[:DEPENDS_ON]→(b:Type:Class)

WITH allClasses, collect(DISTINCT b) AS reusedClasses

RETURN
size(reusedClasses) AS reusedClasses,
size(allClasses) AS totalClasses,
round(100.0 * size(reusedClasses) / size(allClasses)) AS
reuseFactorPercent
```

Production Classes + Test Classes

	reusedClasses	totalClasses	reuseFactorPercent
1	29	42	69.0

```
MATCH (c:Type:Class)
WHERE NOT c.fqn CONTAINS "Test"
WITH collect(c) AS allClasses

MATCH (a:Type:Class)-[:DEPENDS_ON]→(b:Type:Class)
WHERE NOT a.fqn CONTAINS "Test"
AND NOT b.fqn CONTAINS "Test"

WITH allClasses, collect(DISTINCT b) AS reusedClasses

RETURN
size(reusedClasses) AS reusedClasses,
```

Only Production Classes

	reusedClasses	totalClasses	reuseFactorPercent
1	17	29	59.0

Interpretation

69% ⟶ Whole Project, 59% ⟶ Production System

The difference shows that test code introduces additional dependency relationships and therefore increases the apparent level of reuse.

Future-Proof Software Interpretation

Even after excluding test classes:Reuse Factor = 59%, which means:

- More than half of the production classes are reused.
- The application exhibits a reasonable level of modularity.
- Reusable components reduce future implementation effort.
- Reuse supports maintainability and long-term evolution.

Formula: Reuse Factor = Reused Classes / Total Classes

Most Reused Components · Q-2

```
MATCH (a:Type:Class)-[:DEPENDS_ON]→(b:Type:Class)

RETURN
b.fqn AS ReusedComponent,
count(a) AS ReuseCount

ORDER BY ReuseCount DESC
LIMIT 10
```

Production Classes + Test Classes

	ReusedComponent	ReuseCount
1	"de.tud.st.springpetclinic.owner.pet.Pet"	9
2	"de.tud.st.springpetclinic.owner.pet.types.PetType"	9
3	"de.tud.st.springpetclinic.owner.owner.Owner"	9
4	"de.tud.st.springpetclinic.model.BaseEntity"	6
5	"de.tud.st.springpetclinic.model.Vet"	5
6	"de.tud.st.springpetclinic.owner.visit.Visit"	5

```
MATCH (a:Type:Class)-[:DEPENDS_ON]→(b:Type:Class)

WHERE NOT a.fqn CONTAINS "Test"
AND NOT b.fqn CONTAINS "Test"

RETURN
b.fqn AS ReusedComponent,
count(a) AS ReuseCount

ORDER BY ReuseCount DESC
LIMIT 10
```

Only Production Classes

	ReusedComponent	ReuseCount
1	"de.tud.st.springpetclinic.model.BaseEntity"	6
2	"de.tud.st.springpetclinic.owner.owner.Owner"	5
3	"de.tud.st.springpetclinic.medication.services.Contract"	4
4	"de.tud.st.springpetclinic.owner.pet.Pet"	4
5	"de.tud.st.springpetclinic.owner.pet.types.PetType"	3
6	"de.tud.st.springpetclinic.model.NamedEntity"	3

Interpretation:
After excluding test classes, the reuse count decreases. This gives a more realistic view of the actual application architecture. **BaseEntity, Owner, Contract, and Pet** are the most reused production assets.

Including tests: reuse is higher because test classes also depend on domain classes.

Production only: reuse is lower but more meaningful for architecture analysis.

Future-Proof Interpretation:
The production-only result shows that core domain/model classes are reused across multiple application components. This supports:

- modularity
- maintainability
- reduced duplication
- lower future change cost

Methods per Class in PetClinic · Q-3

```
1 MATCH (c:Java:Type)-[:DECLARES]→(m:Method)
2 WHERE c.fqn CONTAINS 'springpetclinic'
3 RETURN c.name AS Class, COUNT(m) AS NumberOfMethods
4 ORDER BY NumberOfMethods DESC
5 LIMIT 10
```

Table | Text | Code

	Class	NumberOfMethods
1	"Owner"	21
2	"OwnerControllerTests"	15
3	"OwnerController"	14
4	"Pet"	12
5	"Vet"	12
6	"Contract"	12
7	"ClinicServiceTests"	12
8	"PetController"	10
9	"Item"	9

INTERPRETATION & FUTURE-PROOF RELEVANCE

- Owner has 21 methods — most complex class, central business entity of the system
- OwnerController (14) & Contract (12) reflect new functionality added in Task 1
- Pet and Vet (12 each) — core domain classes, method count is appropriate
- Test classes appear in top results — indicates test coverage proportional to complexity
- High-method-count classes are highest maintenance risk — first affected by requirement changes
- Potential God Class candidates — should be monitored and refactored as system scales

Fields per Class in PetClinic · Q-4

```
MATCH (c:Type:Class)-[:DECLARES]→(f:Field)
WHERE c.fqn CONTAINS "springpetclinic" AND NOT c.fqn CONTAINS "Test"
RETURN c.name AS Class, count(f) AS Fields
ORDER BY Fields DESC
LIMIT 10
```

Table | Text | Code

	Class	Fields
1	"ContractOverviewDto"	7
2	"Contract"	5
3	"Owner"	5
4	"ContractController"	4
5	"OwnerController"	4
6	"Pet"	3
7		

Started streaming 10 records after 14 ms and completed after 18 ms.

INTERPRETATION & FUTURE-PROOF RELEVANCE

- Counts **fields per class**; how much state each class holds
- **Field counts are low across the board (max 7)**; no data-heavy god classes
- **ContractOverviewDto (7) tops the list; but that's expected**: a DTO exists to bundle data for display
- **Owner and Contract (5 each)** are highest among real domain classes; also top in methods (Q3), so Owner is the main god-class candidate to watch
- **Future-Proof Aspect:** for **Maintainability** low state complexity now prevents god classes later

Coupling Analysis · Q-5

```
MATCH (a:Type:Class)-[:DEPENDS_ON]→(b:Type:Class)
WHERE NOT a.fqn CONTAINS "Test"

RETURN a.fqn,
count(b) AS dependencies

ORDER BY dependencies DESC
LIMIT 10
```

Table | Text | Code

	a.fqn	dependencies
1	"de.tud.st.springpetclinic.owner.owner.Owner"	4
2	"de.tud.st.springpetclinic.medication.services.ContractHandler"	4
3	"de.tud.st.springpetclinic.owner.PetController"	4
4	"de.tud.st.springpetclinic.controller.ContractController"	4
5	"de.tud.st.springpetclinic.owner.OwnerController"	3
6	"de.tud.st.springpetclinic.owner.pet.Pet"	3
7		

Started streaming 10 records in less than 1 ms and completed after 6 ms.

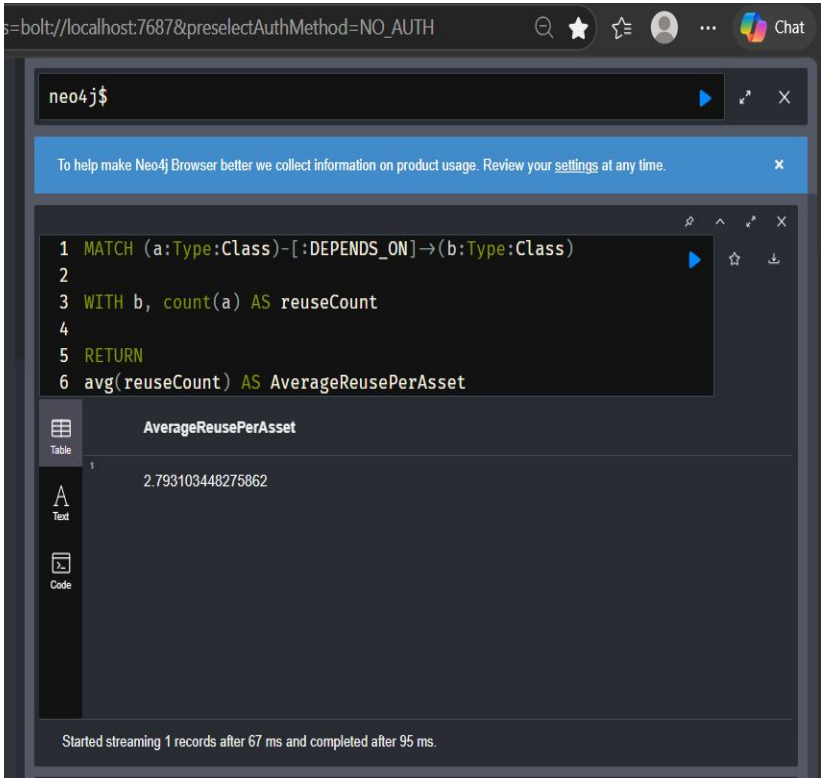
Interpretation

- No class exhibits excessive dependency on other components.
- The dependency structure is **relatively lightweight.**
- Changes are less likely to propagate across many components.
- The architecture demonstrates **moderate coupling**.

Future-Proof Relevance

- Lower coupling improves maintainability.
- Components can **evolve more independently.**
- Changes become **easier to implement and test.**
- Reduced coupling **lowers** the risk of **architecture erosion** over time.

Reuse Cost-Benefit Relation · Q-6



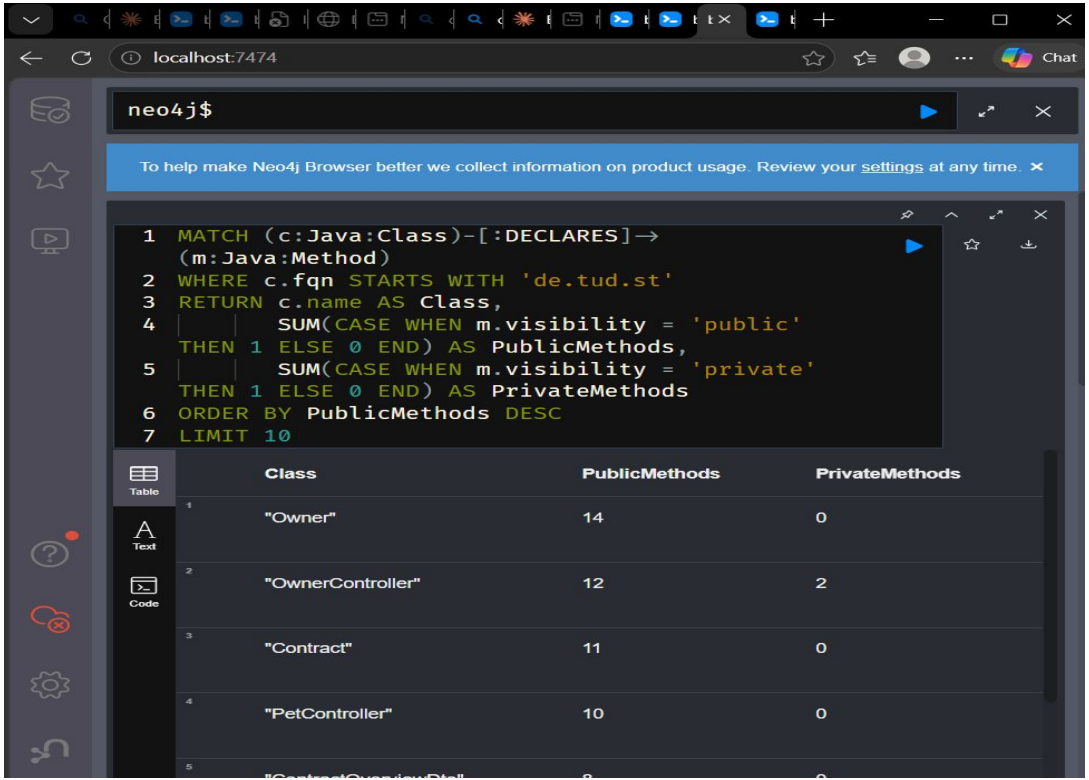
Cost-Benefit Perspective

- Each shared component serves approximately **2.8 consumers** on average.
- Components are reused across multiple parts of the system rather than being single-use.
- Reuse **increases the return on development effort** invested in shared assets.
- Reusable abstractions **reduce redundant** implementation work.
- The benefits of reuse grow as the **system evolves and scales.**

Future-Proof Relevance

- Improves **maintainability** through shared and consistent implementations.
- Reduces duplication and simplifies future modifications.
- Supports **modular software evolution**.
- Enables **lower long-term maintenance costs.**
- Contributes to sustainable and future-proof software architecture.

Visibility of Methods (public vs private) · Q-7



INTERPRETATION & FUTURE-PROOF RELEVANCE

- Measures public vs private method ratio per class to evaluate encapsulation quality
- **8 out of 10 classes have 0 private methods**.
- **Owner** (14 public, 0 private) is the most over-exposed class in the project
- **Contract, Item and UsedItem** — our added classes — also have 0 private methods, making future changes risky
- Only **OwnerController** (2 private) and **ContractHandler** (1 private) show minimal encapsulation

Visibility of Fields (public vs private fields) · Q-8

```
MATCH (c:Type:Class)-[:DECLARES]→(f:Field)
WHERE c.fqn CONTAINS "springpetclinic" AND NOT c.fqn CONTAINS "Test"
RETURN c.name AS Class,
       SUM(CASE WHEN f.visibility = 'private' THEN 1 ELSE 0 END) AS PrivateFields,
       SUM(CASE WHEN f.visibility = 'public' THEN 1 ELSE 0 END) AS PublicFields
ORDER BY PublicFields DESC
LIMIT 10
```

Table | Text | Code

	Class	PrivateFields	PublicFields
1	"Owner"	4	1
2	"Contract"	5	0
3	"Person"	2	0
4	"Pet"	3	0
5	"ContractOverviewDto"	7	0
6	"Visit"	2	0
7			

Started streaming 10 records after 1 ms and completed after 4 ms.

INTERPRETATION & FUTURE-PROOF RELEVANCE

- Measures **field-level encapsulation**; whether data is private (protected) or public (exposed)
- **9 of 10 classes have 0 public fields**
- **Owner exposes one public field, bookedServices** (its list of contracts), not a constant, a real leaked instance field
- Any class can read/modify it directly, so the list can't be validated, logged, or refactored safely later

Inheritance Depth (DIT) · Q-9

```
1 MATCH path = (c:Type:Class)-[:EXTENDS*1..]→(s:Type)
2 WHERE c.fqn CONTAINS "springpetclinic" AND NOT c.fqn CONTAINS "Test"
3 RETURN c.name AS Class, max(length(path)) AS InheritanceDepth
4 ORDER BY InheritanceDepth DESC
5 LIMIT 10
```

Table | Text | Code

	Class	InheritanceDepth
1	"Pet"	3
2	"Vet"	3
3	"Owner"	3
4	"Specialty"	3
5	"PetType"	3
6	"Person"	2
7		

Started streaming 10 records after 12 ms and completed after 35 ms.

INTERPRETATION & FUTURE-PROOF RELEVANCE

- Measures **Depth of Inheritance Tree (DIT)**; how many levels a class sits above its base class; to evaluate hierarchy complexity
- **Maximum depth is only 3** across the whole project; a shallow, healthy hierarchy
- **Pet, Vet, Owner, Specialty, PetType (depth 3)** all inherit through the chain BaseEntity ⟶ NamedEntity ⟶ Person
- Shallow inheritance; a base-class edit ripples to fewer descendants; **easier, safer changes**
- Deep trees (DIT > 5) are a future-proof risk; PetClinic stays well within safe limits

Query Findings & Future-Proof Verdict

Future Proof aspect covered	Queries
Reusability	Q1 (Reuse Factor), Q2 (Most Reused), Q6 (Reuse Cost-Benefit)
Maintainability	Q3 (Methods per Class), Q4 (Fields per Class)
Changeability	Q5 (Coupling), Q9 (Inheritance Depth)
Encapsulation	Q7 (Method Visibility), Q8 (Field Visibility)

Findings

9 queries across 4 future-proof factors; complements SonarQube

1. **Reusability:** 69% → Whole Project, 59% → Production System
2. **Maintainability:** low method/field counts, no god classes
3. **Changeability:** low coupling dependency (max 4), shallow inheritance (depth 3)
4. **Encapsulation:** Fields well protected; but methods are mostly public, more private helper methods would make it more future proof

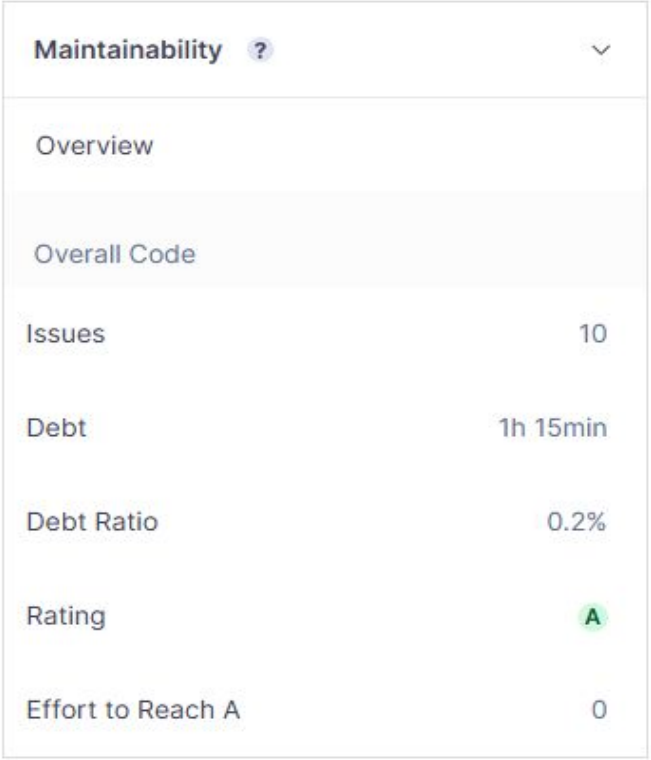
Weaknesses

1. **Methods are mostly public**: 8/10 classes have 0 private methods (Owner: 14 public/0 private)
2. **Leaked field**: Owner.bookedServices is public; editable without validation
3. **Owner can be a god-class candidate**: highest in methods (21) and among the top in fields (5)
4. **Test gaps**: 65.8% coverage; contract feature least covered
5. **Small-project limits**: with a small codebase, metrics reveal less than on a large system

Technical Debt & Maintainability

INTERPRETATION & FUTURE-PROOF RELEVANCE

- Only **1h 15min effort** needed to resolve all identified maintainability issues
- **10 minor code smells** - no bugs, no security vulnerabilities, but TODO comments are reported as maintainability issues.
- A **0.2% debt ratio** means developers spend almost no time fighting the codebase
- Low technical debt reduces future maintenance costs and improves adaptability
- Debt accumulates silently - keeping it low now prevents exponential future cost



Summary & Overall Assessment

SONARQUBE FINDINGS

- Quality Gate: Passed
- Coverage: 65.8%
- Duplication: 0%
- Technical Debt Ratio: 0.2%
- Maintainability Rating: A
- Security Rating: A

JQASSISTANT FINDINGS

- Reuse Factor: 69%
- Average Reuse per Asset: 2.79
- Low production coupling
- Clear and coherent domain model
- Core entities correctly shared

Overall Assessment

The project demonstrates strong reusability, low technical debt, good maintainability, and reasonable test coverage — supporting the principles of Future-Proof Software Systems.